数えるだけの言語モデル

バイグラム ― 学習ゼロで、名前が作れてしまう

rkskmt

前回の振り返り

- ことばは数になった：**stoi** / **itos** / **encode** / **decode** の4点セット
- 地名データ：1,747件、語彙74文字

> **① クイズ**
>
> 地名のよみで、「ょ」の**次**に来る文字は何が多い？ 「きょうと」「ちょうし」…… 予想してから、数えて確かめる。

- 今日のモデルの作戦はこれだけ：**直前の1文字だけ見て、次の1文字を当てる**

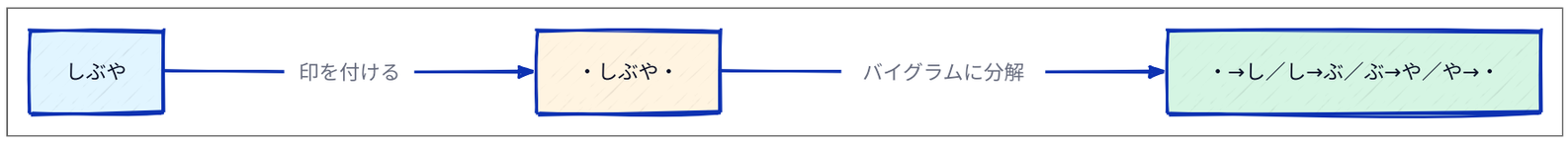
バイグラム ― 隣り合う2文字

- 隣り合う2文字の組を**バイグラム**（bigram）と呼ぶ
- 「しぶや」のバイグラム：「し→ぶ」「ぶ→や」
- データ全体でこの組を**数えるだけ**。学習も勾配もまだ使わない

> **ⓘ ノート**
>
> 「次の1文字を当てる分類器」の最弱バージョン。弱いが、**動くものを最初に作る**のはシステム作りの鉄則。

名前の始まりと終わりに印を打つ

- 「どの文字で始まりやすいか」「どこで終わるか」も当てたい
- そこで名前の前後に境界の印「・」を付けてから数える



- 「・→し」＝「し」で始まった、「や→・」＝「や」で終わった、と読む
- 語彙は74＋1で**75文字**になる

全部数える

Code

```
import numpy as np
import pandas as pd

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = ["・"] + sorted(set("".join(names)))   # 境界を0番にする
stoi = {c: i for i, c in enumerate(chars)}

N = np.zeros((75, 75), dtype=int)
for name in names:
    seq = "・" + name + "・"
    for a, b in zip(seq, seq[1:]):     # 隣り合う2文字を順に取る
        N[stoi[a], stoi[b]] += 1

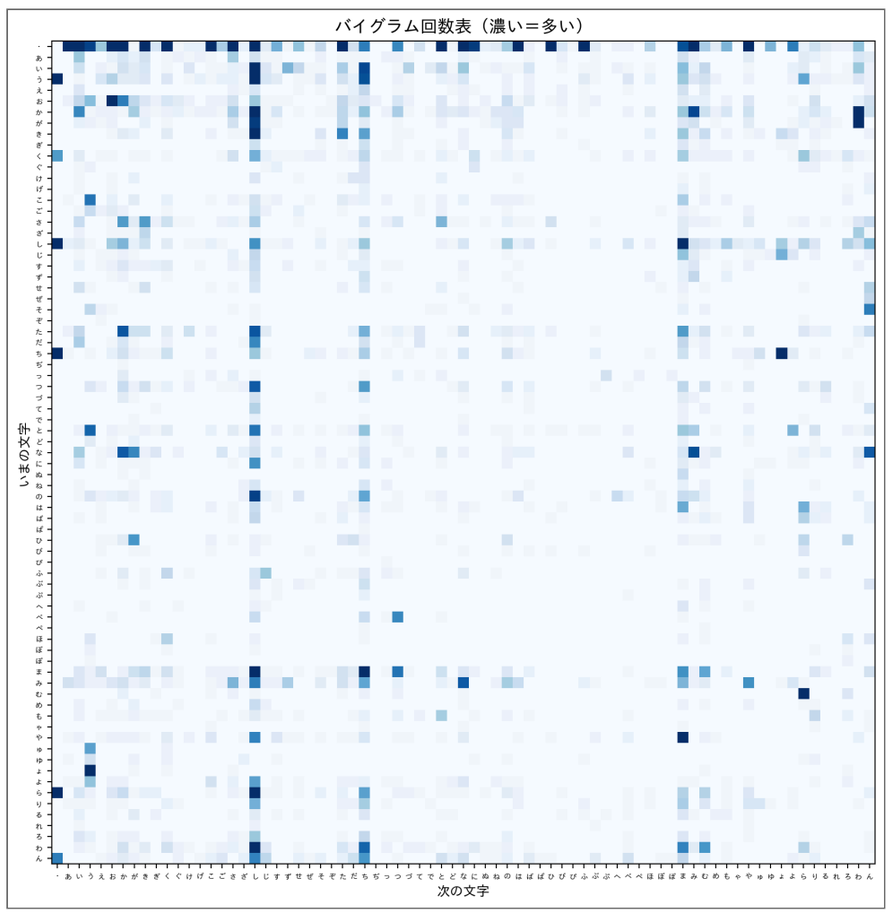
print("組の総数:", N.sum())
print("「ょ→う」の回数:", N[stoi["ょ"], stoi["う"]])
```

Result

```
組の総数: 11428
「ょ→う」の回数: 509
```

- 75×75 の表が「地名らしさ」の全記録。行＝いまの文字、列＝次の文字

表を絵にする



- クリックで拡大して、濃いマスを探検してみる（「ょ」の行、「ん」の列……）

表が知っていること

クイズ

「・」の**行**のトップ5＝名前の最初に来やすい文字。何だと思う？ 「・」の**列**のトップ5＝名前の最後の文字。こっちはかなり偏るはず……

Code

```
import numpy as np
import pandas as pd

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = ["・"] + sorted(set("".join(names)))
stoi = {c: i for i, c in enumerate(chars)}
N = np.zeros((75, 75), dtype=int)
for name in names:
    seq = "・" + name + "・"
    for a, b in zip(seq, seq[1:]):
        N[stoi[a], stoi[b]] += 1

first = N[0].argsort()[::-1][:5]        # ・の行：最初の文字
last = N[:, 0].argsort()[::-1][:5]      # ・の列：最後の文字
print("最初:", [(chars[i], N[0, i]) for i in first])
print("最後:", [(chars[i], N[i, 0]) for i in last])
```

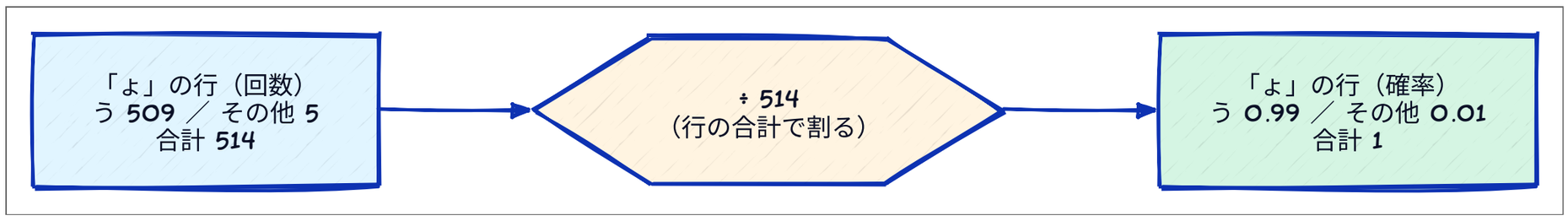
Result

```
最初: [('か', np.int64(135)), ('お', np.int64(123)), ('い', np.int64(106)), ('み',
np.int64(102)), ('し', np.int64(98))]
最後: [('し', np.int64(792)), ('う', np.int64(466)), ('ち', np.int64(277)), ('ら',
np.int64(161)), ('ん', np.int64(28))]
```

- 最後は「し」792回の圧勝 — **市**・町（ちょう／まち）・村の「制度」が表に写り込んでいる
- 誰も教えていないのに、数えただけで「地名の文法」が浮かび上がる

回数を確率に直す

- 行の合計で割れば、その文字の次に来る文字の**確率分布**になる



- 前コースの分類器の出力（10クラスの確率）と同じ形。クラスが75個になっただけ

Code

```
import numpy as np
import pandas as pd

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = ["・"] + sorted(set("".join(names)))
stoi = {c: i for i, c in enumerate(chars)}
N = np.zeros((75, 75), dtype=int)
for name in names:
    seq = "・" + name + "・"
    for a, b in zip(seq, seq[1:]):
        N[stoi[a], stoi[b]] += 1

P = N / N.sum(axis=1, keepdims=True)   # 各行を合計1に
print("P(う|ょ) =", P[stoi["ょ"], stoi["う"]].round(4))
print("P(・|し) =", P[stoi["し"], stoi["・"]].round(4))   # 「し」で終わる確率
```

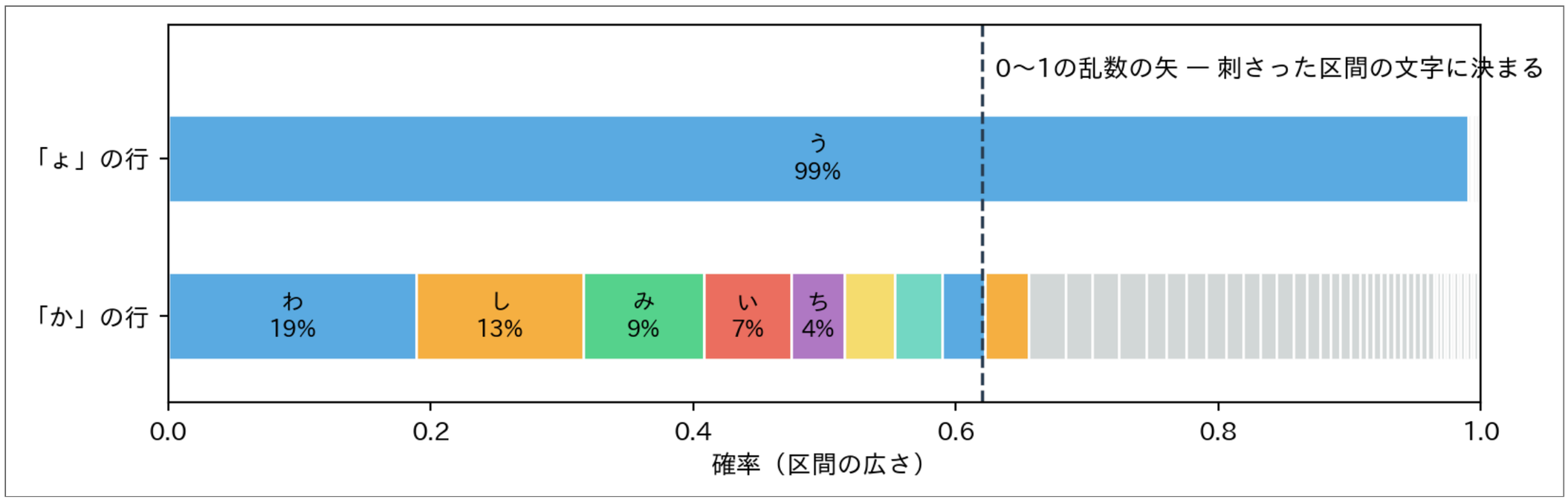
Result

```
P(う|ょ) = 0.9903
P(・|し) = 0.7097
```

- 「ょ」の次は99%「う」。「し」まで来たら71%でそこで終わり

確率で1文字選ぶ

- 分布から**確率に比例して**1文字引く。イメージは**区間の広さが確率に比例したルーレット**



- 「か」のルーレットは わ・し・み…と割れている。「ょ」はほぼ「う」一色 ― **行ごとに違うルーレット**を回す
- これをやってくれるのが **`np.random.choice(..., p=確率)`**

Code

```
import numpy as np
import pandas as pd

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = ["・"] + sorted(set("".join(names)))
stoi = {c: i for i, c in enumerate(chars)}
N = np.zeros((75, 75), dtype=int)
for name in names:
    seq = "・" + name + "・"
    for a, b in zip(seq, seq[1:]):
        N[stoi[a], stoi[b]] += 1
P = N / N.sum(axis=1, keepdims=True)

rng = np.random.default_rng(0)
i = stoi["ょ"]
for _ in range(5):
    print(chars[rng.choice(75, p=P[i])], end=" ")   # 「ょ」の次を5回引く
```

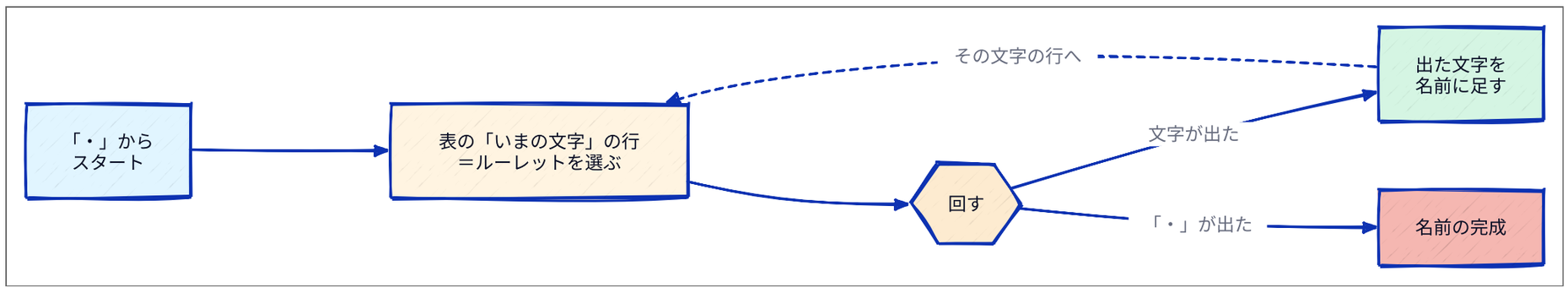
Result

```
う う う う う
```

- ほぼ「う」、たまに別の字 ― サイコロの目が確率表どおりに偏っている

生成 — 表とサイコロだけで名前を作る

- 「・」から始めて、出た文字を次の入力にして繰り返す。「・」が出たら終わり



Code

```
import numpy as np
import pandas as pd

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = ["・"] + sorted(set("".join(names)))
stoi = {c: i for i, c in enumerate(chars)}
N = np.zeros((75, 75), dtype=int)
for name in names:
    seq = "・" + name + "・"
    for a, b in zip(seq, seq[1:]):
        N[stoi[a], stoi[b]] += 1
P = N / N.sum(axis=1, keepdims=True)

rng = np.random.default_rng(0)
for _ in range(10):
    i, s = 0, ""                        # 0 = 「・」からスタート
    while True:
        i = rng.choice(75, p=P[i])
        if i == 0:
            break                       # 「・」が出たら名前の完成
        s += chars[i]
    print(s)
```

Result

```
とさいいままちょうみまいまがわしまち
さいさとべつしんらし
なごうし
きたやまちょう
くぜんごし
まち
みえだいたまし
さがさっさとかわし
とりちょう
とんむらそんだてしかむら
```

出来ばえ品評会

- 「きたやまちょう」「なごうし」 — **実在しないのに、実在しそう**
- 「とんむらそんだてしかむら」 — 村を2回付けるな
- 10件とも実在の1,747件には**入っていない**（コピーではなく新作）

> **ヒント**
>
> 出力を毎回変えたいときは **`default_rng(0)`** の数字（乱数の**シード**（random seed））を変える。 同じシードなら同じ出力 — 実験の再現に必須の道具。

でたらめ名の解剖

- 失敗作を1つ、表に戻って調べる：「とんむらそんだてしかむら」

Code

```
import numpy as np
import pandas as pd

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = ["・"] + sorted(set("".join(names)))
stoi = {c: i for i, c in enumerate(chars)}
N = np.zeros((75, 75), dtype=int)
for name in names:
    seq = "・" + name + "・"
    for a, b in zip(seq, seq[1:]):
        N[stoi[a], stoi[b]] += 1

seq = "・とんむらそんだてしかむら・"
print([f"{a}→{b}:{N[stoi[a], stoi[b]]}" for a, b in zip(seq, seq[1:])])
```

Result

```
['・→と:73', 'と→ん:4', 'ん→む:3', 'む→ら:170', 'ら→そ:3', 'そ→ん:28', 'ん→だ:8', 'だ→て:5',
'て→し:12', 'し→か:18', 'か→む:8', 'む→ら:170', 'ら→・:161']
```

- どの一歩も、データに**実在する**組み合わせ（最少でも3回は出てくる）
- **一歩一歩は正しいのに、全体は迷子** — 直前の1文字しか覚えていないから
- 「むら」と付けた時点で、その前に「そん」と言ったことはもう忘れている

Tweak 表とサイコロで遊ぶ



- **生成のコード** を書き換えて実行

1	`default_rng(0)` → `default_rng(7)` など	別の名前セットが出る（同じ数字なら毎回同じ）
2	`i, s = 0, ""` → `i, s = stoi["ふ"], "ふ"`	「ふ」で始まる名前だけ作れる
3	`rng.choice(75, p=P[i])` → `P[i].argmax()`	サイコロをやめて毎回1位を選ぶと……？

ここで観察

- argmax 版は**毎回同じ1つの名前**に行き着く（何が出るかはお楽しみ）。確率で「揺らす」ことが多様さの源
- 書き出しを変えると、表の**その行**から旅が始まる — 生成は行から行への乗り継ぎ

まとめ

- **数えるだけ**の言語モデル：バイグラム回数表 → 行ごとに確率化 → サイコロで次々引く
- 境界トークン「・」が「始まりやすさ」「終わりやすさ」まで面倒を見る
- 一歩一歩は正しいのに全体は迷子 — **記憶が1文字**しかないのが弱点
- 出力は確率的：シードで再現をコントロールする

現在の設計図　文字番号 → 回数表 ⇨ 確率 → サンプリング → 次の文字

次回予告

この表、実は**勾配降下法でも作れる**。「数える」と「学習する」が同じ答えに行き着く瞬間を見る。 交差エントロピー、久しぶりの再登場。

練習問題

以下の問題を**Pythonコード**で解け。

問題1：ある文字の「次」の分布を棒グラフに

「か」の次に来る文字の確率分布を、確率の高い順に**上位10文字**の棒グラフで表示せよ。

0. **chimei.csv** を **`data/`** に配置し、本編のコードで **`P`** まで作る
1. 「か」の行 **`P[stoi["か"]]`** から上位10個の添字を取る（**`argsort`** を利用）
2. **`plt.bar`** で描画（x軸ラベルは文字、**`import japanize_matplotlib`** を忘れずに）

問題2：名前の最後に来られる文字

74文字のうち、**名前の最後の文字になれるもの**（「・」列の回数が1以上の文字）をすべて挙げよ。何種類あるか（先に予想してから数えること）。

問題3：地名らしさ採点機

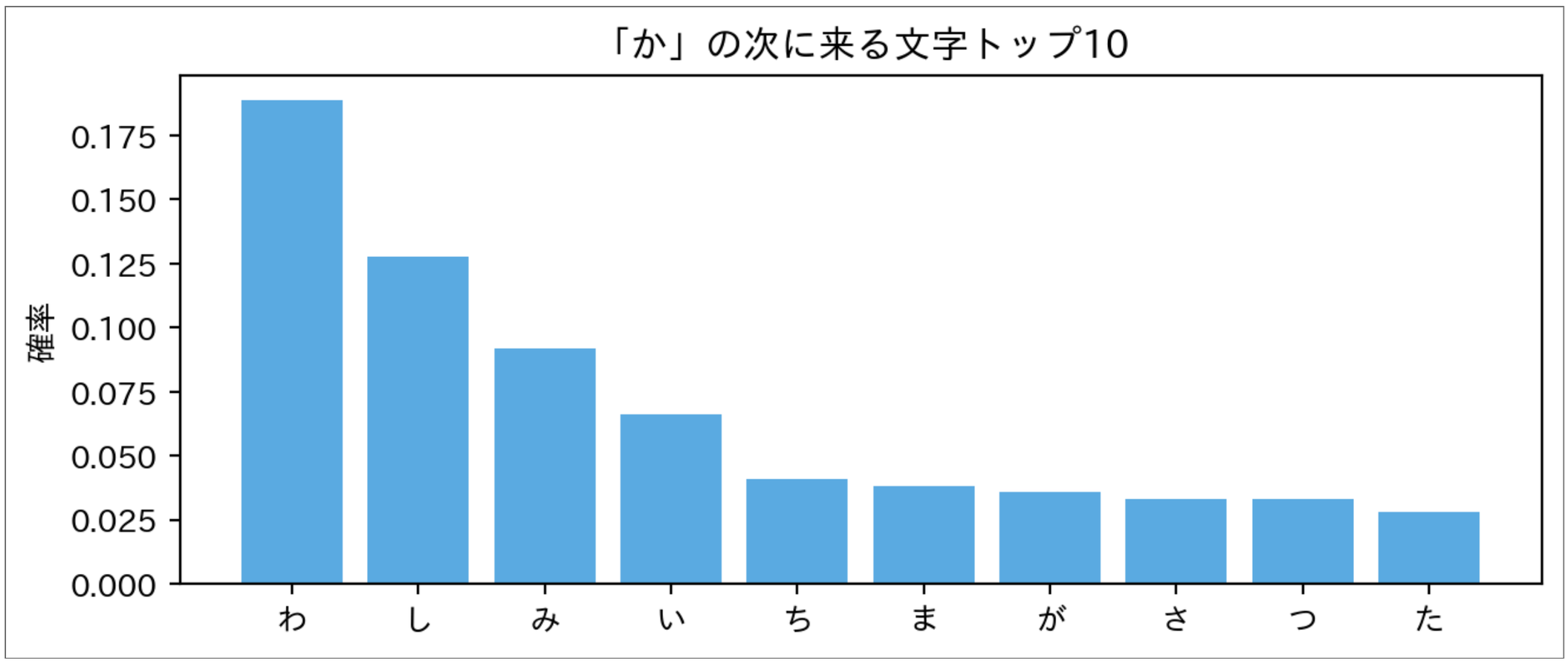
確率表 **`P`** で任意のよみがなの「地名らしさ」を採点する関数を作り、実在の全市区町村名をランキングせよ（採点が甘い＝地名らしい）。

0. **chimei.csv** を **`data/`** に配置し、本編のコードで **`P`** まで作る
1. 関数 **`score(s)`**：「・」＋s＋「・」の隣り合う一歩ごとに $-\log P$ を取り、平均を返す（**`np.log`** を利用）
2. 自作の架空地名を2〜3個採点し、「とうきょう」とスコアを比較（すべて表示）
3. 実在1,747件を全部採点し、**最も地名らしくない実在の地名**トップ5をスコアつきで表示（どんな名前が来るか、先に予想すること）

> **ⓘ ノート**
>
> 表に0のマスを踏むと $-\log 0 = \infty$ — 無限大の罰点。エラーではなく「データに1度もない一歩を踏んだ」という意味。

解答例

▸ 解答例を見る



Result

```
最後に来られる文字（6種類）： ['う', 'く', 'し', 'ち', 'ら', 'ん']
```

- 1,747件の最後の文字は**6種類だけ** — 市・町・村・区の読みが出口を独占している

問題3の解答例

▸ 解答例を見る

Result

```
とうきょう: 2.234
ひがわむら: 1.452
最も地名らしい:
  1.22  かみやまちょう
  1.23  あまちょう
  1.30  おやまちょう
最も地名らしくない:
  3.78  ちよだく
  3.73  ちとせし
  3.57  めぐろく
  3.56  ねりまく
  3.54  ののいちし
```

- 最も地名らしいのは「かみやまちょう」1.22 — 「〜まちょう」は地名の王道中の王道
- 最も地名らしくないのは**『ちよだく』3.78**。千代田区・目黒区・練馬区 — **日本で一番有名な住所たちが「地名らしくない」と採点される**。市町村だらけの全国データで数えた表にとって、都会の短い区名は「珍しい響き」だから
- 「きょうと」を採点すると ∞ — 「と」で終わる名前は1,747件に1つもない（問題2の「6種類」の外側）